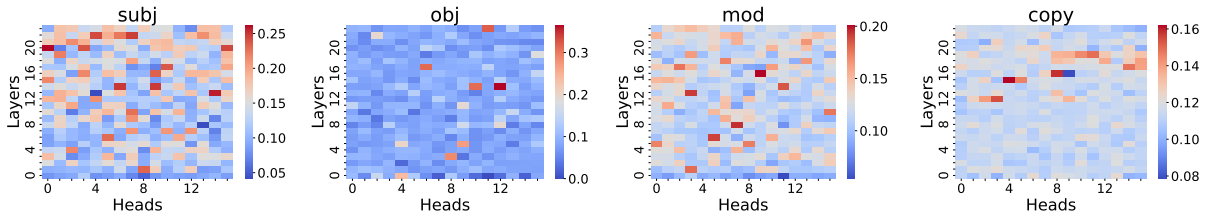

Figure 2: Heatmaps of the average relation index of attention heads for syntactic dependency between tokens and the heatmap of the average copying score (Bansal et al., 2023) of attention heads.

$(t_s, \textit{relation}, t_o)$, *relation* $\in$ {*subj, obj, mod*}. t_s denotes the token in the head node, termed the *head token*, where s denotes its index in the input sequence. t_o represents the token in the child node, termed the *tail token*, where o represents its index in the sequence.

To examine whether attention heads encode dependency relationships between tokens, we use the OV circuit to analyze the influence of attention heads on output logits of tail tokens when attending to head tokens. Given an input sequence $[t_1, \ldots, t_n]$ and the triplet $T = (t_s, \textit{relation}, t_o)$, we measure how much each head h raises the tail token t_o when attending to the head token t_s.

First, we look for attention heads that attend to the head token via the QK circuit. Given the current token t_j, let $A_j^h = \textit{softmax}(x_j W_{QK}^h \boldsymbol{x}^T)$ denote the attention probability of the h-th attention head over all tokens. If the head h attends from t_j to the head token t_s with a high probability, *i.e.,* $s = \arg\max_{1 \le k \le j} A_{j,k}^h$ and $A_{j,s}^h / \max_{k \ne s}\{A_{j,k}^h\} > \tau$, we consider this head as a potential candidate for representing the triplet associated with the head token t_s. Otherwise, we skip this head on this triplet. We set $\tau = 2.2$ in experiments[3].

Second, we examine whether these heads raise the tail token t_o by computing the projection of the OV output on the vocabulary, $x_j W_{OV}^h W_u \in \mathbb{R}^{|\mathcal{V}|}$. Similar to (Bansal et al., 2023), we first compute the output probabilities of tokens as $\boldsymbol{p}^{h,j} = \textit{softmax}(x_j W_{OV}^h W_u) \in \mathbb{R}^{|\mathcal{V}|}$. Then, we extract the probability $p_{t_k}^{h,j} (k \le j)$ of each token t_k before the token t_j . Here we suppose all tokens t_k before t_j are unique for simplification, and if several positions share the same token (*e.g.,* $t_{k_1} = t_{k_2}$), we only consider it once. These probabilities are further transformed by subtracting their mean value and ruling out values smaller than zero, as follows.

$$q_{t_k}^{h,j} = \max(0, p_{t_k}^{h,j} - \mathbb{E}_{1 \le k' \le j}[p_{t_{k'}}^{h,j}]) \qquad (2)$$

[3]Please refer to Appendix A for discussions about the setting of τ.

This transformation helps to focus on tokens whose output probabilities are raised. Then, we compute the following ratio $a_T^{h,j}$ to measure the significance of raising the tail token t_o relative to all tokens before t_j.

$$a_T^{h,j} = q_{t_o}^{h,j} / \sum\nolimits_{k=1}^{j} q_{t_k}^{h,j} \qquad (3)$$

Finally, for each head h, we average the relation index $a_T^{h,j}$ across all current tokens t_j and across all triplets T. Note that we only consider current tokens after the head and tail tokens, *i.e.,* $j \ge \max(s, o)$.

Model's ability in understanding dependencies. Before examining whether attention heads encode dependencies between tokens, we first test the model's overall proficiency in learning and understanding dependency relationships. We follow Clark et al. (2019) to train an attention-and-words probing classifier, which takes the word embeddings and the attention weights extracted from InternLM2-1.8B as input and fits the probability of each token being the syntactic head of another token. We train the classifier on 200 sequences from the AGENDA dataset, each with a length of less than 32. Then, we evaluate the accuracy of the predicted head positions on another 100 sequences. For 52% tokens in the input sequence, the classifier can identify the position of their head tokens based on attention weights extracted from InternLM2-1.8B. This accuracy is significantly higher than the random guess, indicating that InternLM2-1.8B can well understand syntactic dependencies. Therefore, we are motivated to further study dependencies in its attention heads.

The *subj* and *obj* dependencies are encoded in attention heads. Figure 2 shows heatmaps of the relation index of each attention head *w.r.t.* three types of dependency relationships. As a baseline, we also compute the relation index when setting tail tokens in all triplets to the 10th token. Such triplets do not represent any relationships, and the relation

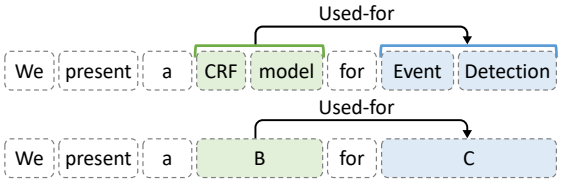

Figure 3: We replace the entities in the sentence (the first row) with capital English letters (the second row).

indexes of attention heads are lower than 0.1. In comparison, for the *subj* and *obj* dependencies, the attention heads exhibit relation indexes close to 0.3, and the relation index *w.r.t.* the *mod* dependency is a bit lower. This suggests the model may have better learned the *subj* and *obj* dependencies than *mod*. In Section 4.1, it is also observed that the model exhibits better performance on the justification of the *subj* and *obj* dependencies than the *mod* dependency, coinciding with this discovery. Furthermore, relation indexes for the *obj* dependency are more sparsely distributed than those for *subj* dependency. There are about ten attention heads that exhibit salient values for the *obj* dependency. This indicates that the model may store the *obj* dependency using a few attention heads, while the *subj* dependency is widely encoded in more attention heads. These observations highlight the varying degrees of the model's understanding and representation of different dependency relationships.

3.2 Semantic Relationship in Attention Heads

In addition to syntactic dependencies, we also study semantic relationships between entities in knowledge graphs. There are seven types of relations between entities in the AGENDA dataset: Part-of, Compare, Used-for, Feature-of, Hyponym-of, Evaluate-for, and Conjunction.

Similar to dependencies in Section 3.1, we also expect to represent each relation between entities as a triplet $T = (t_s, \textit{relation}, t_o)$. Because the entities in sentences often consist of multiple tokens, we replace them with capital English letters[4] as shown in Figure 3. By representing each entity with a single letter, we can directly adopt the metric in Equation (3). Besides, we remove the most frequent function words annotated by spaCy in the sentence. This step helps to reduce noise and focus on the more informative content words.

Various semantic relationships are encoded in attention heads. Figure 4 illustrates relation indexes of attention heads *w.r.t.* seven types of semantic relationships. In contrast to syntactic dependencies, semantic relationships exhibit clearer patterns within attention heads. Each type of relationship is represented by a range of 5 to 15 attention heads. Interestingly, certain relationships, such as "Used-for," "Hyponym-of," and "Conjunction," appear to be more clustered in specific attention heads. These findings suggest that the model possesses a capacity to represent various semantic relationships in attention heads. Furthermore, considering these semantic relationships are bidirectional, we also analyze the reverse relation triplet ($\tilde{T} = (t_o, \textit{relation}, t_s)$) in attention heads in Appendix B. Results in Figure 16 show that some attention heads store both directions of the relationship, reflecting the model's ability to understand the reciprocal nature of these semantic relationships.

4 In-Context Learning and Semantic Induction Heads

In this section, we investigate the correlation between in-context learning and semantic induction heads. We first categorize the ICL ability into three levels and observe the gradual emergence of different levels of ICL. Then, we investigate the formation of semantic induction heads in the training process to understand the emergence of ICL.

4.1 In-Context Learning of Different Levels

In-context learning refers to the ability to learn from the context to perform an unseen task. However, there is no standard measurement for the ICL ability of LLMs. Kaplan et al. (2020); Olsson et al. (2022) consider ICL as the ability to better predict later tokens in the context than earlier tokens. Another more widely adopted definition of ICL follows a few-shot setting. In this setting, language models are provided with a few examples within the context of the prompt, and the model can better perform the task with more examples given. This definition emphasizes the model's ability to extract and generalize the information in the context.

We rethink the ICL ability from the perspective of what the model has learned from the context. The loss reduction considered in (Olsson et al., 2022) only demonstrates that the context does help models make predictions, but it is unclear what the model has learned. Chomsky (1957) has proposed that when humans learn new languages, they initially grasp the surface structure of the language

[4]We exclude special letters like A,I,N,S,W, and E, which often appear alone and are meaningful alone.

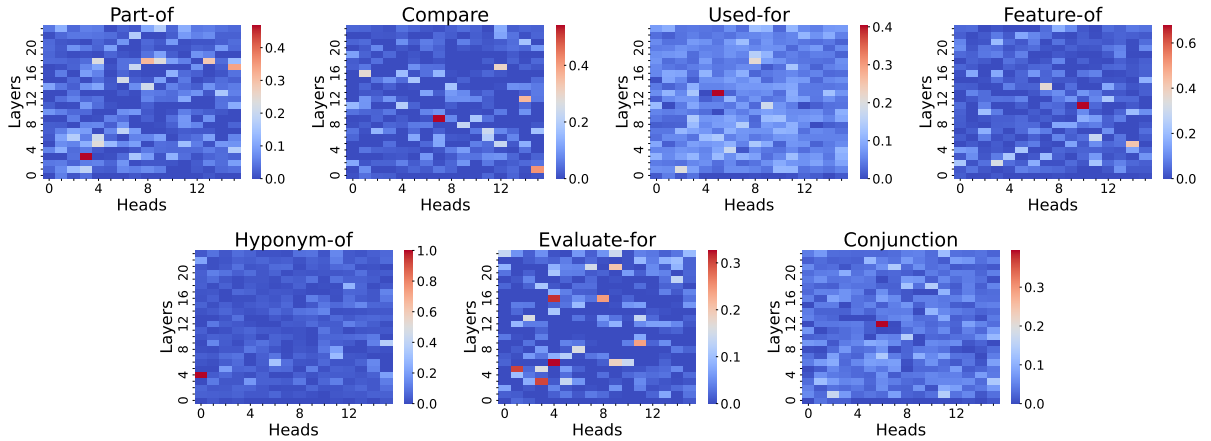

Figure 4: Heatmaps of the average relation index of attention heads for semantic relationships in knowledge graphs.

Task	Entity set	Template
Binary classification	(fruit, month), (furniture, profession)	<E1, E2>: 0; <E2, E1>: 1
Four-class classification	(fruit, month)	<E1, E1>: 0; <E1, E2>: 1; <E2, E1>: 2; <E2, E2>: 3
Nine-class classification	(fruit, animal, month)	<E1, E1>: 0; <E1, E2>: 1; <E1, E3>: 2; <E2, E1>: 3; $\cdots$
Relation justification	(*subj*, *verb*), (*verb*, *obj*), (*mod*, *obj*), (part, whole)	<E1, E2>: true ; <animal, month>: false

Table 1: We construct toy tasks for evaluating the ICL ability of models. E1, E2, and E3 in the template refer to instances belonging to the 1st, 2nd, and 3rd categories in each pair of entity sets. For example, binary classification contains inputs like "apple, January: 0" and "April, orange:1".

before delving into the deep structure. Inspired by this, we hypothesize that LLMs also first learn the surface format of the context, and then gradually comprehend the deep patterns or rules within the context. Based on this hypothesis, we categorize the ICL ability into three levels:

- Loss reduction: This level of ICL is characterized by a reduction in the loss of tokens as the model predicts later tokens in the context. Olsson et al. (2022) demonstrates ICL at this initial level.
- Format compliance (few-shot): At this level, the model learns the format of examples in the prompt (*e.g.,* numbers and symbols), and generates outputs following the same format. Although the outputs have the correct format, the predictions may be incorrect.
- Pattern discovery (few-shot): This level expects the model to recognize and comprehend the underlying pattern within the examples, and apply it consistently to generate the correct prediction.

By categorizing ICL into these levels, we can systematically assess the progression and development of the model's ICL abilities.

Model. To study the ICL ability of the model during the training process, we train a model from scratch using the InternLM framework (Team, 2023). The model contains 20 transformer layers, and each layer consists of $H = 16$ attention heads.

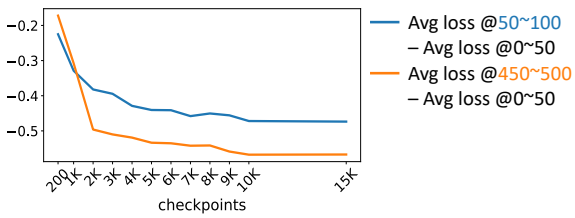

Figure 5: The average difference in loss between later tokens and early tokens decreases from the very beginning of the training.

The hidden size of the model is $d = 2048$, thus each head has a dimension of $d_h = 128$. The model is trained using the SlimPajama dataset (Soboleva et al., 2023) on 32 GPUs, and the batch size on each GPU is 128K tokens. We train the model for 40k steps, with checkpoints saved every 200 steps to monitor the model's progress in relationship representing and ICL during training.

Measurements and results. We assess the ICL ability at each level using the following methods.

For loss reduction, we follow Olsson et al. (2022) but with a minor modification that improves the statistical stability. Specifically, we adjust the formula from the loss at the j-th token minus the loss at the i-th token ($i \leq j$), to the averaged loss over the interval from the $j \sim (i+j)$-th tokens minus the averaged loss over the $0 \sim i$-th tokens.

We sample 100 sentences from the SlimPajama